

Lecture 8: For + While Loops



INFO 1100: Introduction to Programming

Our code can do so many
things now!

Our code can do so many
things now!

**Store
values**

Our code can do so many
things now!

**Store
values**

**Make
decisions**

But what if we want to do
things...

But what if we want to do
things...

many times?

But what if we want to do
things...

many times?



**and we don't
know how many**

But what if we want to do
things...

many times?



**and we don't
know how many**

*x number of
times?*

But what if we want to do
things...

many times?



**and we don't
know how many**

*x number of
times?*

*for all items in a
list?*

But what if we want to do
things...

many times?



**and we don't
know how many**

*x number of
times?*

*for all items in a
list?*

*while something is
true?*

Loops

Loops

```
graph TD; A[Loops] --> B[For]; A --> C[While];
```

A diagram illustrating the classification of loops. The word "Loops" is at the top center. Two curved arrows branch out from below "Loops" to the words "For" and "While". "For" is on the left and "While" is on the right.

For

While

For-Loops

For-Loops

What do they do?

For-Loops

iterate over a collection
of items

For-Loops

```
for <var> in <seq>:  
    <statement>  
    ...
```


For-Loops

```
for <var> in <seq>:  
    <statement>  
    ...
```

- assign the first element of <seq> to <var> and then execute all <statement>s

For-Loops

```
for <var> in <seq>:  
    <statement>  
    ...
```

- assign the first element of <seq> to <var> and then execute all <statement>s
- assign the second element of <seq> to <var> and then execute all <statement>s

For-Loops

```
for <var> in <seq>:  
    <statement>  
    ...
```

- assign the first element of <seq> to <var> and then execute all <statement>s
- assign the second element of <seq> to <var> and then execute all <statement>s
- ...

For-Loops

```
for <var> in <seq>:  
    <statement>  
    ...
```

loop variable

For-Loops

```
for <var> in <seq>:  
    <statement>  
    ...
```

loop sequence

For-Loops

```
for <var> in <seq>:
```

```
<statement>
```

```
...
```

loop body

For-Loops

```
for item in lst:  
    print(item)
```

For-Loops

```
for item in lst:  
    print(item)
```

Print all items in `lst`

For-Loops

```
for char in s:  
    print(char)
```

Print all characters in **s**

For-Loops

How many times will the if statement be executed?

```
def mystery(s):  
    for v in 'aeiou':  
        if v in s:  
            return True  
    return False
```

- a) Exactly 5 times
- b) 1 to 5 times
- c) Exactly len(s) times
- d) 1 to len(s) times
- e) Never

For-Loops

Enter in the chat:

**When would you want to use a
for-loop?**

Range

Range

What if we want to
iterate from 1 to n ?

Range

```
for i in n:  
    print(i)
```

Range

```
for i in n:  
    print(i)
```



Integers are not sequences

Range

A **range** is a sequence
of numbers

Range

`range (x)`

Range

`range (x)`

a sequence from 0 up to x

Range

`range(3)`

`list(range(3))` → `[0, 1, 2]`

Range

`range (y, x)`

a sequence from y up to x

Range

```
range(2, 5)
```

```
list(range(2, 5)) → [2, 3, 4]
```

Range

```
r = range(3)
```

Range

```
r = range(3)
```

```
len(r)      →    3
```

Range

```
r = range(3)
```

```
r.start → 0
```


Range

```
r = range(3)
```

`r.stop` → 3

Range

```
r = range(3)
```

`r[0]` → 0

Range

```
r = range(3)
```

`r[1]` → 1

Range

```
for i in n:  
    print(i)
```

Range

```
for i in range(n):  
    print(i)
```

Range

```
for i in range(len(lst)) :  
    print(str(i+1) + ". " + lst[i])
```

Accumulator patterns

Accumulator patterns

An **accumulator** is a variable a loop uses to help compute a value

Accumulator patterns

```
def avg(lst):  
    num = 0  
    for x in lst:  
        num = num + x  
    return num / len(lst)
```

Accumulator patterns

```
def avg(lst):  
    num = 0  
    for x in lst:  
        num = num + x  
    return num / len(lst)
```

accumulator
value

Accumulator patterns

```
accumulator = <start value>
```

```
for item in sequence:
```

```
    ...
```

```
    accumulator = accumulator <op> item
```

```
    ...
```

While-Loops

While-Loops

What do they do?

While-Loops

repeat while condition
is true

While-Loops

```
while <boolean expression>:  
    <statement>  
    ...
```

While-Loops

```
while <boolean expression>:  
    <statement>  
    ...
```

- check if the <boolean expression> is `True`; execute statements if so

While-Loops

```
while <boolean expression>:  
    <statement>  
    ...
```

- check if the <boolean expression> is `True`; execute statements if so
- check if the <boolean expression> is `True`; execute statements if so

While-Loops

```
while <boolean expression>:  
    <statement>  
    ...
```

- check if the <boolean expression> is `True`; execute statements if so
- check if the <boolean expression> is `True`; execute statements if so
- ...

While-Loops

```
while <boolean expression>:  
    <statement>  
    ...
```

- check if the <boolean expression> is True; execute statements if so
- check if the <boolean expression> is True; execute statements if so
- ...
- <boolean expression> is False

While-Loops

```
def mystery(s):  
    v = 'aeiou'  
    i = 0  
    while i < len(v):  
        if v[i] in s:  
            return True  
        i += 1  
    return False
```

While-Loops

```
def mystery(s):  
    v = 'aeiou'  
    i = 0  
    while i < len(v):  
        if v[i] in s:  
            return True  
        i += 1  
    return False
```

**Beware infinite
loops!**

For vs. While-Loops

For vs. While-Loops

computationally
equivalent!

For vs. While-Loops

computationally
equivalent!

*what's easier
to **write**?*

*what's easier
to **read**?*

For vs. While-Loops

Which do you prefer?

```
def total(nums):  
    s = 0  
    for n in nums:  
        s = s + n  
    return s
```

```
def total(nums):  
    s = 0  
    i = 0  
    while i < len(nums):  
        s = s + nums[i]  
    return s
```

Example 1

```
def count_long_words(words):  
    count = 0  
    for w in words:  
        if len(w) >= 5:  
            count += 1  
    return count  
  
print(count_long_words(["cat", "hippo",  
                        "dog", "zebra"]))
```

Example 2

```
def shout(words):  
    result = []  
    for w in words:  
        result.append(w.upper())  
    return result  
  
print(shout(["hello", "loops"]))
```

Example 3

```
def sum_first_n(n):  
    total = 0  
    for i in range(1, n + 1):  
        total = total + i  
    return total  
  
print(sum_first_n(5))
```

Example 4

```
def digit_sum(n):  
    total = 0  
    while n > 0:  
        total += n % 10  
        n = n // 10  
    return total  
  
print(digit_sum(472))
```

Example 5

```
def first_negative(nums):  
    i = 0  
    while i < len(nums):  
        if nums[i] < 0:  
            return nums[i]  
    return None  
  
print(first_negative([3, 7, -2, 5]))  
print(first_negative([1, 2, 3]))
```

Questions?

(... I'll wait for 2)

For tomorrow:

(07/08 @ 11:30am)

- Lab 8
- Readings
Think Python –
5.8-5.10, 6.6-6.9
- GRQs